Assignment 5

Q1. Docker Swarm: Nginx Service

Deploy an Nginx service in Docker Swarm.

Requirements:

1. Initialize Docker Swarm.
2. Create a service:
   - Service name: `nginx-service`
   - Image: `nginx`
   - Replicas: `2`
3. Expose container port `80` on host port `8080`.
4. Verify the service using:
   - `docker service ls`
   - `docker service ps nginx-service`
5. Access Nginx from the browser.

```
root@server-virtual-machine:~# docker swarm init
Swarm initialized: current node (wnso9chz6mjgrb8gccoo41r3o) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-0ww6yh31fspgm6i3eqln0x95kfv5druf6byiu752h7t0jwdcgg-6uemgx5408ijb868vapv3orh2 192.168.74.123:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
root@server-virtual-machine:~# docker service create \
  --name nginx-service \
  --replicas 2 \
  --publish 8080:80 \
  nginx
ls4zilpfc7yvkku9s0w1axqwi
overall progress: 2 out of 2 tasks
1/2: running
2/2: running
verify: Service ls4zilpfc7yvkku9s0w1axqwi converged
```

```
root@server-virtual-machine:~# docker service ls
ID             NAME            MODE         REPLICAS   IMAGE          PORTS
ls4zilpfc7yv   nginx-service   replicated   2/2        nginx:latest   *:8080->80/tcp
root@server-virtual-machine:~# docker service ps nginx-service
ID             NAME              IMAGE          NODE                      DESIRED STATE   CURRENT STATE            ERROR     PORTS
4wo4x1m611pc   nginx-service.1   nginx:latest   server-virtual-machine   Running         Running 40 seconds ago
z7nvbe87fh07   nginx-service.2   nginx:latest   server-virtual-machine   Running         Running 39 seconds ago
root@server-virtual-machine:~#
```

Q2. Docker Swarm: Python Worker Service

Deploy a Python script as a Docker Swarm service.

Requirements:

1. Create `app.py` that prints:

```
Running in Swarm...
```

every 5 seconds.

1. Create a Dockerfile.
2. Build a Docker image.
3. Deploy the service:
   - Service name: `python-service`
     Replicas: `3`
4. Check logs from different running containers.
5. Verify task distribution using:

```
docker service ps python-service
```

```
root@server-virtual-machine:~/Assignments/Assignment5# nano app.py
root@server-virtual-machine:~/Assignments/Assignment5# nano Dockerfile
```

```
  GNU nano 6.2                                app.py
import time
import sys

while True:
        print("Running in Swarm... ", flush=True)
        time.sleep(5)
```

```
  GNU nano 6.2                              Dockerfile *
FROM python:3.12-slim
WORKDIR /app
COPY app.py
CMD ["python", "app.py"]
```

```
root@server-virtual-machine:~/Assignments/Assignment5# docker build -t python-worker:1.0 .
[+] Building 2.5s (9/9) FINISHED                                docker:default
 => [internal] load build definition from Dockerfile                      0.0s
 => => transferring dockerfile: 111B                                      0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim       2.1s
 => [auth] library/python:pull token for registry-1.docker.io             0.0s
 => [internal] load .dockerignore                                         0.0s
 => => transferring context: 2B                                           0.0s
 => [1/3] FROM docker.io/library/python:3.12-slim@sha256:aa943ce411423c7  0.0s
 => => resolve docker.io/library/python:3.12-slim@sha256:aa943ce411423c7  0.0s
 => [internal] load build context                                         0.0s
 => => transferring context: 128B                                         0.0s
 => CACHED [2/3] WORKDIR /app                                             0.0s
 => [3/3] COPY app.py .                                                   0.1s
 => exporting to image                                                    0.2s
 => => exporting layers                                                   0.1s
 => => exporting manifest sha256:c57820b40671cd86862c32a6bb92deaafa8d74e   0.0s
 => => exporting config sha256:872f23c71878bd2ec346b9bfd473826d422f97c42   0.0s
 => => exporting attestation manifest sha256:b3b150b2c1b853607e508bfa1c4   0.0s
 => => exporting manifest list sha256:a35781160eb777461b518eb001cfaa9f0a   0.0s
 => => naming to docker.io/library/python-worker:1.0                      0.0s
 => => unpacking to docker.io/library/python-worker:1.0                   0.0s
root@server-virtual-machine:~/Assignments/Assignment5# docker service create --name python-service --replicas 3 python-worker:1.0
image python-worker:1.0 could not be accessed on a registry to record
its digest. Each node will access python-worker:1.0 independently,
possibly leading to different nodes running different
versions of the image.

iq0nk7tfizgq6157ppwhys2bo
overall progress: 3 out of 3 tasks
1/3: running
2/3: running
3/3: running
verify: Service iq0nk7tfizgq6157ppwhys2bo converged
```

```
root@server-virtual-machine:~/Assignments/Assignment5# docker service logs python-service
python-service.1.9bcftz7klf0l@server-virtual-machine    | Running in Swarm...
python-service.1.9bcftz7klf0l@server-virtual-machine    | Running in Swarm...
python-service.1.9bcftz7klf0l@server-virtual-machine    | Running in Swarm...
python-service.1.9bcftz7klf0l@server-virtual-machine    | Running in Swarm...
python-service.1.9bcftz7klf0l@server-virtual-machine    | Running in Swarm...
python-service.1.9bcftz7klf0l@server-virtual-machine    | Running in Swarm...
python-service.1.9bcftz7klf0l@server-virtual-machine    | Running in Swarm...
python-service.1.9bcftz7klf0l@server-virtual-machine    | Running in Swarm...
python-service.1.9bcftz7klf0l@server-virtual-machine    | Running in Swarm...
python-service.1.9bcftz7klf0l@server-virtual-machine    | Running in Swarm...
python-service.1.9bcftz7klf0l@server-virtual-machine    | Running in Swarm...
python-service.2.xy1nai6g046d@server-virtual-machine    | Running in Swarm...
python-service.2.xy1nai6g046d@server-virtual-machine    | Running in Swarm...
python-service.2.xy1nai6g046d@server-virtual-machine    | Running in Swarm...
python-service.2.xy1nai6g046d@server-virtual-machine    | Running in Swarm...
python-service.2.xy1nai6g046d@server-virtual-machine    | Running in Swarm...
python-service.2.xy1nai6g046d@server-virtual-machine    | Running in Swarm...
python-service.2.xy1nai6g046d@server-virtual-machine    | Running in Swarm...
python-service.2.xy1nai6g046d@server-virtual-machine    | Running in Swarm...
python-service.2.xy1nai6g046d@server-virtual-machine    | Running in Swarm...
python-service.2.xy1nai6g046d@server-virtual-machine    | Running in Swarm...
python-service.2.xy1nai6g046d@server-virtual-machine    | Running in Swarm...
python-service.3.40b85khof4dj@server-virtual-machine    | Running in Swarm...
python-service.3.40b85khof4dj@server-virtual-machine    | Running in Swarm...
python-service.3.40b85khof4dj@server-virtual-machine    | Running in Swarm...
python-service.3.40b85khof4dj@server-virtual-machine    | Running in Swarm...
python-service.3.40b85khof4dj@server-virtual-machine    | Running in Swarm...
python-service.3.40b85khof4dj@server-virtual-machine    | Running in Swarm...
python-service.3.40b85khof4dj@server-virtual-machine    | Running in Swarm...
python-service.3.40b85khof4dj@server-virtual-machine    | Running in Swarm...
python-service.3.40b85khof4dj@server-virtual-machine    | Running in Swarm...
python-service.3.40b85khof4dj@server-virtual-machine    | Running in Swarm...
python-service.3.40b85khof4dj@server-virtual-machine    | Running in Swarm...
```

```
root@server-virtual-machine:~/Assignments/Assignment5# docker service ps python-service
ID             NAME              IMAGE                NODE                    DESIRED STATE   CURRENT STATE            ERROR     PORTS
9bcftz7klf0l   python-service.1   python-worker:1.0    server-virtual-machine   Running         Running 2 minutes ago
xy1nai6g046d   python-service.2   python-worker:1.0    server-virtual-machine   Running         Running 2 minutes ago
40b85khof4dj   python-service.3   python-worker:1.0    server-virtual-machine   Running         Running 2 minutes ago
```

Q3. Docker Swarm: Scaling Service

Scale an existing Docker Swarm service.

Requirements:

1. Deploy an Nginx service with 2 replicas.
2. Scale the service to 5 replicas.

```
docker service ls
docker service ps nginx-service
```

3. Verify replicas using:

1. Scale the service back to 1 replica.
2. Observe what happens to containers.

```
root@server-virtual-machine:~/Assignments/Assignment5# docker service create --name nginx-service --replicas 2 -p 8086:80 nginx
1ubr3loqhtw4d83tlflig5oa9
overall progress: 2 out of 2 tasks
1/2: running
2/2: running
verify: Service 1ubr3loqhtw4d83tlflig5oa9 converged
root@server-virtual-machine:~/Assignments/Assignment5# docker service scale nginx-service=5
nginx-service scaled to 5
overall progress: 5 out of 5 tasks
1/5: running
2/5: running
3/5: running
4/5: running
5/5: running
verify: Service nginx-service converged
```

```
root@server-virtual-machine:~/Assignments/Assignment5# docker service ls
ID             NAME             MODE         REPLICAS   IMAGE               PORTS
1ubr3loqhtw4   nginx-service    replicated   5/5        nginx:latest        *:8086->80/tcp
iq0nk7tfizgq   python-service   replicated   3/3        python-worker:1.0
root@server-virtual-machine:~/Assignments/Assignment5# docker service ps nginx-service
ID             NAME               IMAGE          NODE                      DESIRED STATE   CURRENT STATE
xqi6okq326a2   nginx-service.1    nginx:latest   server-virtual-machine    Running         Running about a minute ago
1eqy7w0i3pzk   nginx-service.2    nginx:latest   server-virtual-machine    Running         Running about a minute ago
o9fy1u17vyvg   nginx-service.3    nginx:latest   server-virtual-machine    Running         Running 36 seconds ago
qsn6bgpnlbww   nginx-service.4    nginx:latest   server-virtual-machine    Running         Running 36 seconds ago
4fs14033w20n   nginx-service.5    nginx:latest   server-virtual-machine    Running         Running 36 seconds ago
root@server-virtual-machine:~/Assignments/Assignment5# docker service scale nginx-service=1
nginx-service scaled to 1
overall progress: 1 out of 1 tasks
1/1: running
verify: Service nginx-service converged
root@server-virtual-machine:~/Assignments/Assignment5#
```

Q4. Docker Swarm: Rolling Update

Perform a rolling update in Docker Swarm.

Requirements:

1. Deploy a service using image `nginx:1.25`.
2. Update the service to use image `nginx:latest`.
3. Use rolling update options:

- Update one container at a time
- Delay between updates: 10 seconds

4. Verify update status.
5. Roll back the service if needed.

```
root@server-virtual-machine:~/Assignments/Assignment5# docker service rm nginx-service
docker service create --name nginx-service --replicas 3 -p 8086:80 nginx:1.25
nginx-service
8hk7dv9j75seg00ardz9j9tou
overall progress: 3 out of 3 tasks
1/3: running
2/3: running
3/3: running
verify: Service 8hk7dv9j75seg00ardz9j9tou converged
root@server-virtual-machine:~/Assignments/Assignment5# docker service update \
> --image nginx:latest \
> --update-parallelism 1 \
> --update-delay 10s \
> nginx-service
nginx-service
overall progress: 3 out of 3 tasks
1/3: running
2/3: running
3/3: running
verify: Service nginx-service converged
```

```
root@server-virtual-machine:~/Assignments/Assignment5# docker service inspect --format '{{json .UpdateStatus}}' nginx-service
{"State":"completed","StartedAt":"2026-10-30T09:02:06.797393877Z","CompletedAt":"2026-10-30T09:03:01.897395204Z","Message":"update completed"}
root@server-virtual-machine:~/Assignments/Assignment5# docker service ps nginx-service
ID             NAME                IMAGE          NODE                    DESIRED STATE   CURRENT STATE            ERROR     PORTS
v5aorqyra6fq   nginx-service.1     nginx:latest   server-virtual-machine   Running         Running 4 minutes ago
tj4ayd1fj17g    \_ nginx-service.1   nginx:1.25     server-virtual-machine   Shutdown        Shutdown 4 minutes ago
nj42xw9wh1r4   nginx-service.2     nginx:latest   server-virtual-machine   Running         Running 3 minutes ago
zyyu5ol0n0y4    \_ nginx-service.2   nginx:1.25     server-virtual-machine   Shutdown        Shutdown 3 minutes ago
uk7umfzmkntk   nginx-service.3     nginx:latest   server-virtual-machine   Running         Running 3 minutes ago
w4hueqgghilk    \_ nginx-service.3   nginx:1.25     server-virtual-machine   Shutdown        Shutdown 3 minutes ago
root@server-virtual-machine:~/Assignments/Assignment5# docker service update --rollback nginx-service
nginx-service
rollback: manually requested rollback
overall progress: rolling back update: 3 out of 3 tasks
1/3: running
2/3: running
3/3: running
verify: Service nginx-service converged
root@server-virtual-machine:~/Assignments/Assignment5#
```

Q5. Docker Swarm: Remove Service and Leave Swarm

Clean up Docker Swarm resources.

Requirements:

1. List all running services.
2. Remove `nginx-service`.
3. Remove `python-service`.
4. Verify services are deleted.
5. Leave Docker Swarm mode.

```
root@server-virtual-machine:~/Assignments/Assignment5# docker service ls
ID             NAME             MODE         REPLICAS   IMAGE                PORTS
8hk7dv9j75se   nginx-service    replicated   3/3        nginx:1.25           *:8086->80/tcp
iq0nk7tfizgq   python-service   replicated   3/3        python-worker:1.0
root@server-virtual-machine:~/Assignments/Assignment5# docker service rm nginx-service python-service
nginx-service
python-service
root@server-virtual-machine:~/Assignments/Assignment5# docker service ls
ID        NAME      MODE      REPLICAS   IMAGE     PORTS
root@server-virtual-machine:~/Assignments/Assignment5# docker swarm leave --force
Node left the swarm.
root@server-virtual-machine:~/Assignments/Assignment5#
```

Q6. Explain the usage of Docker Swarm.

Cover the following points:

1. Container orchestration
2. Service deployment
3. Scaling containers
4. Load balancing
5. Rolling updates
6. High availability
7. Multi-node container management

---------->

Usage and Benefits of Docker Swarm

1. **Container Orchestration**: Automates the lifecycle, deployment, management, and coordination of multi-container architectures across a cluster of physical or virtual engines.

---

2. **Service Deployment**: Allows developers to declare a desired target configuration state (image version, network configs, replica volumes) via declarative commands or stack files, which Docker continuously keeps operational.
3. **Scaling Containers**: Facilitates instantaneous scaling of workloads upward or downward using commands like *docker service scale*, adapting swiftly to traffic spikes.
4. **Load Balancing**: Features an integrated ingress routing mesh. Swarm automatically exposes service ports to all public cluster entry points and balances incoming request traffic dynamically across available active task backends.
5. **Rolling Updates**: Executes zero-downtime application updates by progressively deploying software upgrades across nodes while ensuring a threshold of active instances remains responsive.
6. **High Availability**: Features native self-healing features. If an underlying machine crashes or a container goes offline, Swarm automatically provisions matching instances on healthy operational nodes.
7. **Multi-Node Container Management**: Pools multiple distinct Docker hosts into a single abstract computing interface, letting developers orchestrate resources without needing to connect individually to each separate node.

Q7. Explain the limitations of Docker Swarm.

Cover the following points:

1. Less powerful than Kubernetes
2. Smaller ecosystem
3. Limited auto-scaling support
4. Limited networking features
5. Limited storage management
6. Not widely used in large production systems
7. Fewer monitoring and security integrations

--------->

Limitations of Docker Swarm

1. **Less Powerful than Kubernetes**: Lacks highly advanced orchestration tools, Custom Resource Definitions (CRDs), complex scheduling constraints, and complex state management frameworks natively present in Kubernetes.
2. **Smaller Ecosystem**: The community size, library collections, third-party plugin marketplace, and developer support framework are significantly narrower compared to the dominant Kubernetes environment.
3. **Limited Auto-Scaling Support**: Lacks native horizontal pod auto scaling (HPA) driven by memory or CPU performance metrics out of the box; adjustments typically require manual script execution or external extensions.
4. **Limited Networking Features**: While overlay networks work well for simple traffic routing, Swarm lacks native, highly advanced ingress routing controllers, granular network policies, and Service Mesh integration.
5. **Limited Storage Management**: Persistent volume configuration across multi-cloud environments is rigid. It provides less mature volume binding provisioning options compared to the Kubernetes Container Storage Interface (CSI).
6. **Not Widely Used in Large Production Systems**: Enterprises managing massive, multi-tenant global infrastructures generally prefer Kubernetes due to Swarm's limitations in running ultra-large scale processing nodes efficiently.
7. **Fewer Monitoring and Security Integration**: Lacks out-of-the-box integration for modern service meshes, granular secret management tools, or automated enterprise-grade telemetry visualization pipeline.